

1 Lesson 09-10-26

1.1 Introduction to ARM architectures

ARM (which means anything from *Acorn RISC Machine* to *Advanced RISC Machine*) is a british company specialized in RISC processor designs (sold as **IP**, *Intellectual Property*).

1.1.1 Timeline

A short timeline is:

- 1985: ARM1, first ARM processor (v1), designed by Acorn Computers produced by VLSI Technology (now NXP);
- 1987-1989: ARM2 and ARM3 (v2), used by Acorn in its desktop PCs, sold by VLSI as part of processor chip sets;
- 1990: Acorn and Apple found ARM Ltd. with a new business model: manufacturing rights instead of processors. Of course, VLSI become the first licensee;
- 1991: ARM6 (v3);
- 1993: ARM7 (v4), ARM7TDMI: Debug and ICE (In-Circuit Emulation) plus a new compressed instruction set: **Thumb**;
- 1996-2006: ARM8 - ARM11 (v5, v6);
- 2004-today: Cortex cores (v7, v8, v9).

Today the 3 main product lines mantained by ARM are:

- **Cortex-A, X**: high-end applications (smartphones)
- **Cortex-R**: real-time and safety-critical applications
- **Cortex-M**: microcontrollers (Cortex-M3 LPC1768)

1.1.2 Busines model

ARM has seldom delivered stand-alone devices with its own microprocessor. It mainly sells cores, to be used for integration in microcontrollers or *Systems on Chip* (**SoCs**).

Two types:

- **Hard cores**: ARM provides a physical layout optimized for a specific semiconductor technology
- **Soft cores**: ARM provides a high-level description that the designer can synthesize for a specific technology.

1.1.3 Microprocessor vs microcontrollers

Let's look at the difference between **microprocessors** (which we are used to) and **microcontrollers**. A microcontroller is a single computer system packed into a computer chip, which can in turn be placed in a board or some other device, to control it as an *embedded system*.

A common format is **SoC**, which we said means *System-on-Chip*. An SoC is fundamentally an entire system integrated in a single piece of silicon. Most microcontrollers are examples of SoC integration technologies, but SoCs are widely used in other applications too.

To summarize:

- Microprocessor describes a processor-oriented chip;
- Microcontroller describes a computer optimized for control, typically with integrated memory and peripherals;
- SoC describes the integration of a system's major components into one chip.

1.1.4 ARM v7-M architecture

Let's give a quick overview of the ARM v7-M architecture, which is the architecture used in the ARM Cortex-M3 processor we will be using.

Data types The following data types are supported:

- byte: 8 bits
- halfword: 16 bits
- word: 32 bits

General registers Registers are 32-bit wide.

We have the following general-purpose registers (some aliased):

Register	Synonym	Use
r0	a1	general purpose registers
r1	a2	
r2	a3	
r3	a4	
r4	v1	
r5	v2	
r6	v3	
r7	v4	
r8	v5	
r9	v6, SB	
r10	v7	
r11	v8	
r12	IP	
r13	SP, MSP, PSP	stack pointer
r14	LR	link register
r15	PC	program counter

We use the aliases in the table. The main ones are:

- **IP**: not really significant, means *Intra-Procedure scratch* for compiler generated routines (see AAPCS below);
- **SP**: this is the *Stack Pointer*, with **MSP** and **PSP** being respectively the *Main* (handler) and *Program* (thread/user) stack pointers in the respective modes;
- **LR**: this is the *Link Register* for jump and link calling convention;
- **PC**: the good old program counter.

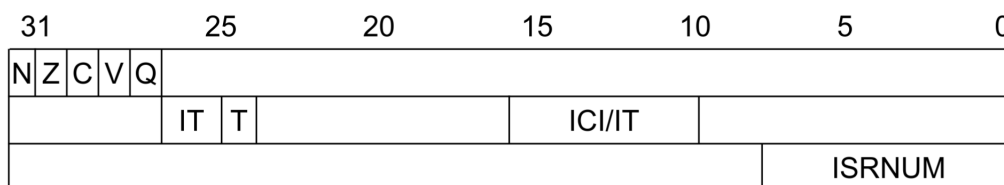
Arm defines **AAPCS** (*ARM Architecture Procedure Call Standard*) for the caller convention ABI. A quick overview is:

- Registers from r0 to r3 are the first 4 function arguments (from the fifth onwards we use the stack). Register r0 then takes the return value. These are *caller-saved*;
- Registers from r4 to r11 are *callee-saved* scratch;
- Register r12 is, as we said, a special *intra-procedure scratch*. The only significant characteristic for now is that it's *caller-saved*, as the first four.

Special registers We also have the following special registers:

Special register	Use
xPSR	program status register
PRIMASK	interrupt mask registers
FAULTMASK	
BASEPRI	
CONTROL	control register

The **PSR** (*Program Status Register*) is significant.



It can be accessed all at once (PSR) or as a combination of 3 registers (in order, the three rows of the figure above):

- *Application Program Status Register (APSR)*: This contains application info (akin to the status flags on x86):
 - N: Negative result from ALU;
 - Z: Zero result from ALU;
 - C: ALU operation Carried out;
 - V: ALU operation oVerflowed;
 - Q: “sticky” flag for saturation arithmetic.
- *Execution Program Status Register (EPSR)*. This contains program execution status usually not of interest to the programmer:
 - IT: IF-THEN instruction status bits. This is interesting: the ARMv7-M (Cortex-M3) uses conditional execution to let instructions execute only when a condition is true. The IT instruction controls this for Thumb instructions;
 - ICI: Interrupt-Continuable Instruction bits;
 - T: Thumb bit. It is always 1: clearing this bit will cause a hard fault exception in ARM-v7.
- *Interrupt Program Status Register (IPSR)*. It contains an exception number used in exception handling. The ISRNUM is extremely important as, if it is different than 0, signals that the processor is in *handler* mode (the equivalent of system mode in ARM).

Instructions There are 3 kinds of instructions:

- ARM instructions: 32 bits wide

- Thumb instructions: 16 bits wide
- Thumb-2 instructions: 16 or 32 bits wide

In fact, ARM v7-M architecture supports Thumb and Thumb-2 instructions only.

Processor modes We have orthogonal operating and privilege modes.

There are two operating modes:

- Thread mode: on reset or after completing the exception routine;
- Handler mode: when an exception occurs.

And two access levels:

- User level: limited access to resources;
- Privileged level: access to all resources.

Note that handler mode is always privileged.

The CONTROL register has only 2 bits, which control:

- Bit 0, which is **nPRIV**, controls the access level;
- Bit 1, which is **SPSEL**, controls the current stack (main or process).

The CONTROL register can be changed only in privileged mode. When an exception is taken, the processor automatically switches to Handler mode. When the handler finishes and exception return occurs, the processor returns to Thread mode if no other exception handler should run. This means handler state is encoded in $IPSR \neq 0$.

We show a summarizing table:

	privileged level	user level
handler mode	CONTROL[0] = 0 CONTROL[1] = 0	not allowed
thread mode	CONTROL[0] = 0 CONTROL[1] = 0/1	CONTROL[0] = 1 CONTROL[1] = 0/1

Interrupt Vector Table ARM-v7 has an **IVT** (*Interrupt Vector Table*).

- Each entry manages an exception, interrupt, or other atypical events such as a reset;
- The vector table contains either ARM instructions or addresses in memory: in the v7-M architecture, there are addresses.